

LeetCode 160 – Intersection of Two Linked Lists ★★★★★

Concept (Optimal - Two Pointer Head Switching)

- Start two pointers at `headA` and `headB`.
- When a pointer reaches the end, redirect it to the other list's head.
- After traversing both lists, they will either:
 - Meet at the intersection node.
 - Both become `nullptr` (no intersection).

Time: `O(m + n)`

Space: `O(1)`

Edge Cases

- No intersection.
- One list is empty.
- Both heads are the same.
- Intersection at the last node.

Concept Code Snippet (Optimal)

```
ListNode* a = headA;
ListNode* b = headB;

while (a != b) {
    a = (a) ? a->next : headB;
    b = (b) ? b->next : headA;
}

return a; // Intersection node or nullptr
```

Pattern to Remember

Pointer A → List A → List B

Pointer B → List B → List A

They meet after traversing equal total distance.

Alternative 1 – Hash Set

Concept

- Store all nodes of one list in a hash set.
- Traverse the second list.
- The first node already in the set is the intersection.

Time: `O(m + n)`

Space: `O(m)`

Concept Code Snippet

```
unordered_set<ListNode*> st;

while (headA) {
    st.insert(headA);
    headA = headA->next;
}

while (headB) {
    if (st.count(headB))
        return headB;
    headB = headB->next;
}

return nullptr;
```

Alternative 2 – Length Difference

Concept

- Find lengths of both lists.

- Advance the longer list by the length difference.
- Move both pointers together until they meet.

Time: $O(m + n)$

Space: $O(1)$

Concept Code Snippet

```
int lenA = length(headA);
int lenB = length(headB);

while (lenA > lenB) {
    headA = headA->next;
    lenA--;
}

while (lenB > lenA) {
    headB = headB->next;
    lenB--;
}

while (headA != headB) {
    headA = headA->next;
    headB = headB->next;
}

return headA;
```

Interview Tip

The **Two-Pointer Head Switching** approach is the expected solution because it achieves $O(m + n)$ time and $O(1)$ space without explicitly computing lengths.

LeetCode 138 – Copy List with Random Pointer ★★★★★

Concept (Optimal - Node Interleaving)

- Insert a copy of each node **right after the original node**.
- Assign random pointers using:
 - $copy \rightarrow random = original \rightarrow random \rightarrow next$
- Separate the original and copied lists.

Time: $O(n)$

Space: $O(1)$ (excluding the copied list)

Edge Cases

- Empty list.
- $random == nullptr$.
- $random$ points to itself.
- Multiple nodes point to the same random node.

Concept Code Snippet

1. Insert Copy Nodes

```
Node* curr = head;

while (curr) {
    Node* copy = new Node(curr->val);

    copy->next = curr->next;
    curr->next = copy;

    curr = copy->next;
}
```

2. Set Random Pointers

```
curr = head;

while (curr) {
    if (curr->random)
        curr->next->random = curr->random->next;

    curr = curr->next->next;
}
```

3. Separate the Two Lists

```
Node dummy(0);
Node* copyTail = &dummy;

curr = head;

while (curr) {
    Node* copy = curr->next;

    curr->next = copy->next;
    copyTail->next = copy;
    copyTail = copy;

    curr = curr->next;
}

return dummy.next;
```

Pattern to Remember

Interleave Copy Nodes → Copy Random Pointers → Separate the Two Lists

```
Node copyRandomList(Node head) {
    if (head == null) return null;

    Node curr = head; // Create new list w/ same values
    while (curr != null) {
        Node newNode = new Node(curr.val);
        newNode.next = curr.next;
        curr.next = newNode;
        curr = newNode.next;
    }

    curr = head; // Copy the random pointers
    while (curr != null) {
        if (curr.random != null)
            curr.next.random = curr.random.next;
        curr = curr.next.next;
    }

    curr = head; // Separate the two lists
    Node newHead = head.next;
    Node newCurr = newHead;
    while (curr != null) {
        curr.next = newCurr.next;
        curr = curr.next;
        if (curr != null) {
            newCurr.next = curr.next;
            newCurr = newCurr.next;
        }
    }

    return newHead;
}
```

DRY-RUN

Alternative – Hash Map

Concept

- Create a copy of every node and store the mapping:
- `original → copy`.
- Traverse again to set `next` and `random` using the map.

Time: `O(n)`

Space: `O(n)`

Concept Code Snippet

```
unordered_map<Node*, Node*> mp;
```

```
for (Node* cur = head; cur; cur = cur->next)
    mp[cur] = new Node(cur->val);

for (Node* cur = head; cur; cur = cur->next) {
    mp[cur]->next = mp[cur->next];
    mp[cur]->random = mp[cur->random];
}

return mp[head];
```

Interview Tip

The **interleaving (weaving)** approach is the one interviewers usually expect because it avoids the extra **$O(n)$** hash map while still running in **$O(n)$** time.

```
*****
**
```

LRU cache will be done in a separate list with all the other related problems